

lakearch — Das Datenmodell

REGELWERK. VOLLSTÄNDIG, DOMÄNEN-FREI, TECHNOLOGIE-FREI.

Präambel. Dieses Dokument beschreibt das Datenmodell *lakearch* erschöpfend und allgemein. Es nennt keine Domäne, keine Anwendung, keine Technologie. Es ist als Axiomensystem gefasst: wenige Grundbegriffe, daraus eindeutig folgende Regeln. Jede Regel gilt ausnahmslos; wo eine Ausnahme nötig schiene, ist die Regel falsch gefasst.

Notation. A , B , K bezeichnen beliebige Daten. „ $A \triangleright K$ “ liest sich „ A besitzt den Kontext K “. Beispiele sind rein abstrakt; lakearch kennt keine Inhalte.

§1 Geltungsbereich

1.1 lakearch **speichert** Daten und ihre Kontexte.

1.2 lakearch **traversiert** entlang von Kontexten — vorwärts wie rückwärts.

1.3 lakearch **matcht strukturell**: Gleichheit auf Inhalts-/Adressebene, „zeigt ein Kontext auf ein Daten?“, Zugehörigkeit zu einer per Kontext gegebenen Menge.

1.4 lakearch **deutet nicht** (Inhalte bleiben opak, §2.1; Bedeutung entsteht erst durch das Vokabular der Ableitung, §14), **rechnet nicht** (keine Arithmetik, Ordnung, Aggregation), **wertet nicht** (erzeugt keine Konfidenz, entscheidet keine Identität, validiert keine Eingabe) und **sortiert nicht**. lakearch ist ein **passiver Speicher**: es hält Daten, ohne eigene Logik über sie auszuführen.

1.5 Alles Rechnen, Werten und Sortieren liegt in einer **Schicht über lakearch**. Sie liest per Traversierung und schreibt ihre Ergebnisse als Daten zurück.

1.6 **Zyklen sind erlaubt**. Der Graph ist kein DAG; ein Kontext darf mittelbar auf sein eigenes Besitzer-Daten zurückzeigen. Strukturelle Terminierung folgt nicht aus der Form, sondern aus der Art der Traversierung (§1.7).

1.7 Traversierung tritt in zwei Sorten auf:

(a) **Mechanische Traversierung** — das Zugriffs-Tor (§11) und die Invalidierung (§10) — ist **deterministisch, beschränkt** (über eine Besuchsmenge) **und damit zyklensicher**. Sie matcht nur; sie rechnet und wertet nicht.

(b) **Explorative Traversierung** — Suche, Platzierung, absichtliches Zurückgehen — liegt in der Schicht darüber (§1.5). Sie darf Zyklen nutzen und ist nicht an Determinismus gebunden.

§2 Die Entität

2.1 Es gibt genau **eine** Art von Entität: das **Daten**. Alles, was existiert, ist ein Daten.

2.2 Es gibt keine zweite Entitätsklasse — kein Schema, keine Metadaten-, Versions-, Zeit- oder Typ-Objekte als eigene Art. Diese Reduktion ist das Kernprinzip.

2.3 Ein **Bestand** ist eine zusammenhängende Menge von Daten unter genau einem Zugriffs-Tor (§11) — eine lakearch-Instanz. Alle folgenden Regeln gelten zunächst *innerhalb* eines Bestands; mehrere verbundene Bestände behandelt §12. Ein **Bereich** (§11) ist demgegenüber eine *logische* Trennung *innerhalb* eines Bestands: **Bestand $\supset$ Bereich**.

§3 Der Kontext

3.1 Ein **Kontext** ist keine eigene Entität, sondern eine **Rolle**: ein Daten K, das von einem Daten A besessen wird. K selbst ist ein gewöhnliches Daten.

3.2 Besitz ist **gerichtet**: „ $A \triangleright K$ “ ist nicht symmetrisch.

3.3 Ein **Verweis** von A auf B wird ausgedrückt, indem A einen Kontext besitzt, der auf B zielt. Die Art des Verweises (die Relation) ist selbst ein Daten.

3.4 Ein Kontext kann selbst Kontexte besitzen. **Reifikation ist kostenlos**: Aussagen über Aussagen sind nativ.

3.5 Jede Aussage des Systems ist ein Daten, das von einem Daten besessen wird. Eine andere Form von Aussage existiert nicht.

3.6 Ein Verweis zeigt **stets auf ein vorhandenes Daten**. Ein noch nicht eingetroffenes Ziel wird durch ein **Platzhalter-Daten** dargestellt — ein gewöhnliches Daten mit einem Kontext, der es als *unaufgelöst/erwartet* ausweist. Spätere Aufnahme ersetzt es (§6.3) oder verbindet sich über den Korrelations-Pfad (§5.7 b). So bleibt jeder Verweis geschlossen: es gibt keine baumelnden Verweise, nur explizit als unaufgelöst markierte Daten.

§4 Typ

4.1 **Typ** ist ein besonderer Kontext: ein Daten trägt seinen Typ als Kontext, der auf ein Typ-Daten zeigt.

4.2 Da ein Typ ein Daten ist, gibt es **keinen Meta-Typ-Regress**; Typ-Aussagen sind gewöhnliche Daten.

4.3 Daten haben keine vorgeschriebene Struktur. Struktur entsteht allein durch Kontexte. Jede Domäne ist abbildbar, **ohne das Modell zu erweitern**.

§5 Identität

5.1 Identität ist **keine intrinsische Eigenschaft**, sondern eine Aussage über Daten — und damit selbst ein Kontext.

5.2 **Speicher-Identität**: Jedes je geschriebene Daten trägt eine opake ID, vorzugsweise einen **Inhalts-Hash**. Sie adressiert; sie urteilt nicht.

5.3 **Wert-Identität**: Inhaltsgleiche Daten sind dasselbe Daten — Inhaltsadressierung dedupliziert automatisch. Ein primitives Daten existiert genau einmal.

5.4 **Referenzielle Identität** („dasselbe Ding?“) wird ausschließlich über Kontexte ausgedrückt, niemals durch destruktive Operationen.

5.5 Referenzielle Identität ist **gradiert**: eine Familie von Identitäts-Kontexten unterschiedlicher Stärke (z. B. *deckungsgleich*, *ergänzt*, *widerspricht-in*, *verwandt-mit*, *bekannt-verschieden*). Jeder trägt eigene Kontexte: betroffene Attribute, Konfidenz, Urheber, Zeit.

5.6 Eigenschaften einer Quelle (stabile Schlüssel, bekannte Abweichungen, Vertrauensgrad) sind Kontexte an dem Daten, das die Quelle repräsentiert.

5.7 Die Aufnahme eines Daten folgt genau einem von drei Pfaden, **erkennbar allein an Kontexten**:

(a) **Fortschreibung** — gleiche Quelle, stabiler Schlüssel, kein Widerspruch: neuer Zustand, der alte wird als ersetzt markiert.

(b) **Korrelation** — andere Quelle oder kein eindeutiger Schlüssel: immer eigenständiges Daten, zuzüglich gradierter Identitäts-Kontexte.

(c) **Konflikt** — keine Richtung trägt oder es bestehen aktive Widersprüche: als solcher modelliert; Auflösung auf die Leseseite verschoben (§8).

5.8 Welcher Pfad gilt, **wählt die schreibende Schicht (§7)** — nicht lakearch. lakearch *matcht* die Vorbedingungen (existiert ein stabiler Schlüssel? existiert ein Widerspruchs-Kontext?, §1.3); die *Wahl* und jede Wertung trifft die Schicht darüber (§1.4).

§6 Zeit

6.1 Zeit ist Daten. Zeitpunkte und Zeiträume sind Daten; Zeit-Aussagen sind besondere Kontexte.

6.2 Es gibt **zwei Zeitachsen**: **Aufzeichnungszeit** (wann das System es erfuhr) und **Gültigkeitszeit** (wann der Sachverhalt in der Welt gilt). Sie dürfen auseinanderfallen.

6.3 Das Modell ist **append-only**: neues Wissen kommt als neue Daten hinzu. Ein **Ersetzungs-Kontext** markiert Älteres als überholt, ohne es zu löschen.

6.4 Eine „Version“ ist nichts Gespeichertes, sondern eine **Leseregel** (§8).

§7 Das Schreiben

7.1 Die **einzige Mutation** ist **append**: genau ein neues Daten samt seinen Kontexten kommt hinzu. Es wird **nie geändert und nie gelöscht**. Ersetzung (§6.3) und der Aktiv-Marker (§13) sind die *einzigsten* Formen von „Änderung“ — beide selbst nur weitere Schreibvorgänge.

7.2 **Was** geschrieben wird und **wohin** es sich per Kontext bindet, entscheidet die **Schicht über lakearch** (§1.5): sie traversiert zum Orientieren (§1.7 b), *matcht* zum Prüfen (§1.3) und schreibt dann **einmal**. lakearch *findet den Platz nicht und beurteilt die Verbindung nicht*; es nimmt das fertige Ergebnis auf.

7.3 **Append rechnet und wertet nicht** (§1.4). Auch die Entscheidungskriterien der schreibenden Schicht liegen als Daten im Bestand; sie zu *lesen* ist Traversierung, sie *anzuwenden* ist Sache der Schicht darüber. Daraus folgt der Leitsatz: *aller Zustand liegt in lakearch, alle Berechnung darüber — auch die Berechnung, die entscheidet, wohin neuer Zustand kommt*.

7.4 Betrifft ein Schreibvorgang mehrere Daten, ist sein Ergebnis erst wirksam, wenn der Aktiv-Marker es freigibt (§13); bis dahin gelten die Teile als inaktiv.

§8 Auflösung beim Lesen

8.1 Was gradierte Identität (§5), Zeit (§6) und Konflikte bedeuten, wird **beim Lesen** entschieden, nicht beim Schreiben.

8.2 Eine Abfrage wählt implizit: Konfidenz-Schwellen, Quell-Gewichtung, Zeitpunkt und Zeitachse.

8.3 Dieselbe Frage an denselben Bestand kann je nach Wahl verschiedene Ergebnisse liefern. Mehrdeutigkeit wird beim Lesen ehrlich behandelt, nicht beim Schreiben verborgen.

8.4 Lesen erzeugt nichts im Speicher; es **projiziert** aus dem vorhandenen Bestand. Historische Zustände werden so rekonstruiert.

§9 Strukturen der Identitäts-Auflösung

lakearch hält die Strukturen; das Auflösen selbst (Schwellen, Entscheidungen) geschieht in der Schicht darüber (§1.5).

9.1 **Repräsentantensystem**: Tragen mehrere Daten dieselbe referenzielle Identität, existiert ein eigenes **Anker-Daten** (die Klasse). Die Repräsentanten verweisen per Mitgliedschafts-Kontext auf den Anker.

9.2 Anderes verweist auf den **Anker**, nicht auf einen einzelnen Repräsentanten. Ein Repräsentant wird **niemals** in den Anker umgewandelt (§6.3).

9.3 Mitgliedschaft ist **gradiert und revidierbar** — ein Kontext, kein destruktiver Zusammenschluss.

9.4 Ein **Split** entsteht durch neue Kontexte, die alte ersetzen; betroffene Repräsentanten werden re-verwiesen.

9.5 **Kuratierung** fügt ausschließlich reversible Kontexte hinzu (*verbergen, ersetzen, Mitgliedschaft*); die Leseseite filtert sie. Es wird nie gelöscht; physisches Entfernen ist Compaction (§15).

§10 Materialisierung

10.1 Ein berechnetes Ergebnis darf als Daten gespeichert werden; ein neueres ersetzt das ältere (§6.3).

10.2 Ein berechnetes Daten trägt seine **Herkunft als Kontext** — die Bindung an die Eingaben, aus denen es entstand.

10.3 Ändert sich eine Eingabe (auch rückwirkend, §6), werden die betroffenen Ergebnisse durch **Rückwärts-Traversierung** der Herkunfts-Kontexte gefunden (Invalidierung; mechanisch, §1.7 a). Das Neu-Berechnen liegt außerhalb (§1.5).

§11 Zugriff

11.1 **Beschreibung.** Ein **Bereich** ist ein Daten; Zugehörigkeit ist ein Kontext (ein Daten kann mehreren Bereichen angehören). Ein Bereich trennt *logisch innerhalb* eines Bestands (§2.3). Eine **Berechtigung** ist ein Daten mit Kontexten *Subjekt, Bereich, Recht, Zeit, Urheber* — auditierbar und bitemporal.

11.2 **Durchsetzung.** Jeder Lesevorgang passiert ein **unumgebares Tor**. Das Tor bildet die Sichtbarkeit aus den Berechtigungen (reines strukturelles Matching: *Bereich* ∈ *gewährte Bereiche*, §1.3; mechanisch, §1.7 a) und wendet sie **vor jeder Auflösung** an.

11.3 **Filtern vor Auflösen** ist zwingend: nicht-sichtbare Daten werden entfernt, bevor referenzielle Identität (§9) aufgelöst wird — damit nichts Nicht-Sichtbares in ein erlaubtes Ergebnis einfließt.

11.4 **Entzug** ist ein neuer Berechtigungs-Kontext, der den alten ersetzt; künftige Lesevorgänge filtern ihn, bereits Gelesenes bleibt (§6.3).

11.5 Das Tor ist Teil von lakearch — *immer aufgerufen, manipulationssicher, minimal*. Es matcht nur (§1.3); es rechnet nicht.

§12 Föderation

12.1 Ein einzelner Bestand **und** mehrere verbundene Bestände sind beides möglich — mit denselben Regeln.

12.2 **Innerhalb eines Bestands** trennen Bereiche (§11) logisch.

12.3 **Über Bestände hinweg** tragen inhaltsgleiche Daten denselben Inhalts-Hash (§5.2) und sind damit bestand-übergreifend identisch — automatischer Abgleich. Referenzielle Identität wird über den Korrelations-Pfad (§5.7 b) verbunden; einen Bestand in einen anderen aufzunehmen *ist* dieser Pfad.

12.4 Inhalts-IDs sind bestand-unabhängig; **Anker-IDs (§9.1) sind bestand-lokal** und werden beim Zusammenführen über gradierte Identität versöhnt.

12.5 **Beispiel (abstrakt).** Ein Bestand mit zwei Bereichen B_1, B_2 trennt zwei Sichten *logisch* unter einem Tor (§11). Zwei *getrennte* Bestände S_1, S_2 verschmelzen anders: inhaltsgleiche Daten fallen über den Inhalts-Hash automatisch zusammen (§5.2), referenziell „dasselbe Ding“ wird über gradierte Identitäts-Kontexte des Korrelations-Pfads (§5.7 b) verbunden, bestand-lokale Anker (§9.1) werden dabei versöhnt. Beides nutzt nur die schon genannten Primitive — kein Sonderfall.

§13 Atomarität

13.1 Jeder Datenzugriff — **lesend wie schreibend** — ist **atomar**: unteilbar und ohne beobachtbaren Zwischenzustand. Kein Zugriff beobachtet je einen halb-vollzogenen Schreibvorgang; er sieht den Bestand entweder vollständig davor oder vollständig danach.

13.2 Für den **einzelnen Schreibvorgang** ist Atomarität strukturell gegeben: die einzige Mutation ist *append* (§7.1), und ein Daten wird nie geändert und nie gelöscht. Das neue Daten samt Kontexten wird als Ganzes sichtbar oder gar nicht; ein teil-geschriebenes Daten ist unbeobachtbar.

13.3 Für das **einzelne Lesen** folgt Atomarität aus der Unveränderlichkeit: Lesen projiziert (§8.4) aus bereits vorhandenen, unveränderlichen Daten und beobachtet keinen von einem nebenläufigen Schreiben erzeugten Zwischenzustand.

13.4 Für das **Lesen über mehrere Daten** — Traversierung, Abfrage — folgt Atomarität aus dem impliziten **Zeitschnitt** (§8.2) zusammen mit der Append-only-Unveränderlichkeit (§7.1): Der Lesevorgang projiziert (§8.4) gegen genau einen Zeitpunkt. Da Schreiben allein *append* ist und Bestehendes nie ändert, wächst der Bestand nur; ein nebenläufig angehängtes Daten liegt hinter dem Schnitt und bleibt unbeobachtet. So sieht auch der zusammengesetzte Lesevorgang den Bestand vollständig vor oder vollständig nach einem nebenläufigen Schreiben (§13.1) — nie einen Mischzustand.

13.5 Für den **mehrere Daten betreffenden Umbau** (Zusammenführen/Spalten §9, Berechtigungs-Wechsel §11) entsteht gemeinsame Sichtbarkeit durch ein einziges abschließendes **Aktiv-Schreiben**. Bis der Aktiv-Marker gesetzt ist, gelten die Teile als inaktiv; Traversierung ignoriert sie (§7.4).

13.6 Ein halb-vollzogener Umbau ist damit unsichtbar, bis er vollständig ist — **Atomarität ohne Transaktions-Maschinerie**. Nebenläufigkeit über dieses Muster hinaus ist Umsetzungssache (§15).

§14 Ableitung & Trennung

14.1 Eine **domänenspezifische Ableitung** erweitert das Modell nicht. Sie ist Daten *innerhalb* eines lakearch-Bestands: Vokabular (Typ- und Relations-Daten), Konventionen, Konfiguration.

14.2 **Trennungsregel**. Vokabular gehört in die Ableitung. Ein neues Speicher-, Traversier- oder Match-Primitiv gehört in lakearch und muss domänen-frei sein. Rechnen, Werten und Sortieren gehört in die Schicht darüber (§1).

14.3 lakearch kennt **keine** Ableitung namentlich. Es stellt nur Primitive bereit, in denen sich jede Ableitung vollständig ausdrückt.

§15 Bewusst offen (Implementierung & Betrieb)

Diese Punkte sind kein Modell-Bestandteil, sondern Umsetzungs-Entscheidungen:

- **ID-Schema** (Inhalts-Hash / UUID / hybrid), **Serialisierung, Abfrage- und Traversier-Sprache**.
- **Compaction**: wann Überholtes oder Verborgenes *physisch* entfernt werden darf (Spannung Append-only ↔ Lösch-Pflichten).
- **Indexierung & Leistung** von Bereichs-Filter, Anker-Auflösung, Herkunfts-Rückwärts-Traversierung.
- **Nebenläufigkeit** über das Aktiv-Marker-Muster (§13) hinaus.

Anhang A — Abgrenzung zu RDF

Geteilte Intuition: es gibt nur eine Sorte Ding, und Beziehungen sind selbst von dieser Sorte. Unterschiede: **Besitz statt Tripel-Symmetrie** (§3.2); **native Reifikation** (§3.4); **native gradierte Identität** als Repräsentation (§5.5); **native Bitemporalität** (§6.2); **keine Literal/Resource-Unterscheidung** — alles ist Daten (§2.1). Konzeptionell näher an append-only, bitemporalen Modellen (Datomic/XTDB) als an RDF.

Anhang B — Wesen in einem Satz

lakearch ist ein append-only Substrat aus genau einer Entität — Daten, die andere Daten als Kontext besitzen —, das **speichert, traversiert und strukturell matcht** (Typ, Identität und Zeit sind besondere Kontexte; Identität ist gradiert, bitemporal und über Inhalts-Hash förderierbar; Anker, Materialisierung und ein unumgehbare Zugriffs-Tor sind native Strukturen; Schreiben ist allein *append*, während das Finden des Platzes außerhalb liegt; Verweise sind geschlossen — Platzhalter statt Lücke —, Zyklen erlaubt, jeder Zugriff ist atomar — Umbauten werden durch einen Aktiv-Marker gemeinsam sichtbar), während **Rechnen, Werten und Sortieren bewusst außerhalb** liegen.